THE WITNESS — Environment Kit Roadmap (8 weeks)

Prepared 9 Nov 2026 · Repo: billfan157-pixel/the-witness (v0.6.0, Three.js r187) · Basis: code review of the rendering pipeline, world builder and build tooling, plus archaeological and license research.

1. Goal, scope and assumptions

Goal. Replace the box-and-cylinder scenery with an authored, reusable *Environment Kit*: PBR materials, a terrain system, modular architecture, rocks and plants, and hero props. Proof of success is two scenes that look like the cover art, not like prototypes.

Vertical slice (the only scenes that must be finished in 8 weeks):

- **Q01 "Bến không chờ người về"** (Caesarea harbor) — the first scene every player sees.
- **L04 Galilee shore** (Giêsu at the lake, Ga 21) — exercises a second material family (basalt), water, boats and nets.

Everything else (Jerusalem, L01 Exodus, L02 En Gedi, L03 cistern, L05 prison) is *kit-ready but not dressed* by the end of this roadmap. The kit is built so those scenes become assembly work, not new modelling.

Assumptions (change these and the schedule changes):

- Solo developer, part-time, about 15–20 hours per week, so roughly 120–160 hours in total.
- Blender is learnable; no paid software is required.
- The game stays on the web (Three.js). A later Unreal port is possible, and glTF + PBR assets carry over.
- Existing gameplay must not break: `npm test` and `npm run test:browser` stay green.

Out of scope for these 8 weeks: new character art, mocap, WebGPU migration, Temple Mount, full Legends dressing, cinematic VFX rework (the L01 pillar of fire and parted sea keep their current shaders).

2. Starting point: what the repo tells us

These findings drive the plan.

Finding	Where	Consequence for the kit
The engine bundle has no `GLTFLoader`, `KTX2Loader`, `MeshoptDecoder`, `DRACOLoader`, `GTAOPass`, `CSM`, `DecalGeometry` or `BufferGeometryUtils`	`vendor/adventure-3d.js`	Week 1 must add loaders without breaking the bundle
Bundled Three.js is r187 and npm `three@0.187.0` is the latest release and contains every needed addon	engine bundle; npm registry	Pin addons to exactly 0.187.0 so they match the bundle
`BatchedMesh` and `InstancedMesh` already exist in the core	engine bundle	Use them for repeated modules and vegetation

Finding	Where	Consequence for the kit
Buildings are ~20 stacked boxes created by `house()`; boats use lathe + boxes; the lighthouse is stacked cylinders	`witness-world.js`	These are the first things the kit replaces
`height(x,z)` ray-casts the ground mesh for every placed object	`witness-world.js` `decorate()`	Too slow for thousands of instances; build a heightfield sampler
Collision uses circle `solids` registered per building; interaction targets must not get solids	`house()`, `applySolids()`	Kit pieces need collision footprints in metadata and must keep NPC/prop coordinates stable
Materials are procedural shader patches (`AdventureVisuals.surface`) with no authored texture sets	`vendor/adventure-visuals.js`	Add real PBR maps; keep the shader hook only for dust/wear overlays
`build.cjs` inlines only `.jpg`/`.webp` into the standalone HTML, and the service worker pre-caches every file at install	`tools/build.cjs`	Models and KTX2 textures need per-region packs and runtime caching, or the offline build bloats
Single ground plane (156 m wide), playable area about 78 × 66 m per scene	`witness-world.js`	Budgets are per scene, which keeps them tractable
Console warnings: `noSurface` / `noCast` are passed into the `MeshStandardMaterial` constructor	`mat()` in `witness-world.js`	Strip these options before constructing materials

3. Look Bible and evidence rules (Week 0)

Your documents already use three evidence tiers: **[K]** Scripture text, **[V]** archaeology, **[D]** artistic decision. Apply the same tags to every kit asset in a catalog column, so nothing generic from a stock library sneaks into a first-century scene.

Target look. Use `assets/journey-cover.webp` as the north-star image: warm low sun, weathered limestone, dry grass, muted earth palette, atmospheric depth, tactile material detail. Build a one-page reference board with 6–8 stills for each region.

Material families per region (each is a decision, tagged [V] where research supports it):

Region / scene	Primary materials	Key facts to respect
Caesarea harbor (Q01)	Pale local **kurkar** sandstone and limestone ashlar, imported marble accents, lime plaster, hydraulic	Herodian dressing: flat projecting central bosses framed by narrow chiselled margins, with the margin about 8 cm wide and sunk

Region / scene	Primary materials	Key facts to respect
	concrete (opus caementicium) on the moles, timber, rope	about 2 cm. Warehouses on the moles were barrel-vaulted. The harbor was Sebastos, built 22–15 BCE. [V]
Galilee (L04, Capernaum-type village)	**Black basalt**, coarse and largely undressed, laid without mortar; light-coloured plaster; roofs of light timber beams with thatch and mud	Houses are arranged around courtyards with stone stairs to the roof and ovens; walls are thin; a basalt quay wall ran along the shore. The 1st-century Kinneret boat is about 8.2 m × 2.3 m and built from mixed reused timbers. [V]
Jerusalem (later)	Pale limestone, Herodian ashlar, narrow stepped streets	Reuse Caesarea masonry with different weathering
L01 Exodus	Sand, dry earth, goat-hair tents, reeds, cart wreckage	Mostly nature and soft materials [D]
L02 En Gedi	Cliff limestone, caves, springs, thorny scrub	Rock kit and cave pieces [D]
L03 cistern, L05 prison	Cut limestone, wet stone, timber, iron	Interior pieces; keep existing gameplay layout

Palette rule. No neon or modern saturation; earth tones only, as in your character research (madder red, woad blue, ochre, undyed linen and wool). Sacred Tier presence is shown through light, not extra props.

Do not mix regions. A basalt wall must not appear in Caesarea; a European cobblestone texture must not appear anywhere. The catalog column "Region lock" enforces this.

4. Technical standards (write these into `docs/environment-kit-standards.md`)

Item	Standard
Units and axes	1 Blender unit = 1 m; Y up on export; pivot at bottom centre of each module
Module grid	Architecture on a 0.5 m grid; wall modules 2 m and 4 m wide, 3 m and 4.5 m tall
Texel density	Terrain 256 px/m; architecture and props 512 px/m; hero props up to 1024 px/m
Texture sets	Albedo (sRGB), normal, ORM packed (occlusion / roughness / metal). Source 2K, ship 1K–2K as KTX2 (UASTC for normals, ETC1S for albedo/ORM)

Item	Standard
Triangle budgets	Wall module ≤ 800; column ≤ 1,500; small prop ≤ 3,000; hero prop ≤ 12,000; tree LOD0 ≤ 8,000; rock ≤ 2,500
LODs	LOD0 + LOD1 (~40%) for everything over 1,000 tris; distant billboards for trees
Materials	One material per module (trim-sheet or atlas) so draw calls stay low
Vertex colour	R = baked ambient occlusion, G = wear/dust mask, B = wind weight (vegetation)
Collision	glTF `extras.footprint` holds circles `[x, z, r]`; imported automatically into `solids`
Naming	`ENV_<region>_<type>_<variant>_LOD0`, e.g. `ENV_CAE_Wall_Ashlar_4m_A_LOD0`
Licence	Every external asset gets a row in `assets/ART-CREDITS.md`: source URL, licence, date, modifications

Proposed per-scene budgets (targets to verify with profiling, not measured facts): desktop ≤ 1.2 M visible triangles, ≤ 250 draw calls, ≤ 256 MB GPU texture memory; "balanced" mobile ≤ 400 k triangles, ≤ 120 draw calls, ≤ 128 MB.

5. Toolchain and asset sources

Purpose	Tool / source	Notes
Modelling, UV, bake	Blender (free)	Use Geometry Nodes for stone-block and thatch generators; bake AO to vertex colour
Optimise glTF	`@gltf-transform/cli` (4.5.1 on npm)	dedup, weld, simplify, meshopt, texture resize
Texture compression	KTX-Software `toktx`	Produces the KTX2 files `KTX2Loader` reads
Loader bundle	`esbuild` + `three@0.187.0` addons	`GLTFLoader`, `KTX2Loader`, `meshopt_decoder.module.js`, `DecalGeometry`, `BufferGeometryUtils`, optionally `GTAOPass`, `CSM`, `SMAAPass` (all verified present in the 0.187.0 package)
CC0 textures, models, HDRIs	**Poly Haven**, **ambientCG**	Poly Haven states all assets are CC0 and usable commercially; ambientCG is CC0 too. Both are safe to ship, but still record each in the ledger
Scanned assets	**Fab / Megascans**	The Fab Standard License allows use in any engine, but pricing is per asset and the EULA has

Purpose	Tool / source	Notes
		distribution limits. Read it before shipping extractable files in a browser game
One-off models	Sketchfab	Per-model licence; accept only CC0 or CC-BY with credit; never "editorial"
Blockout only	Kenney, Quaternius	Stylised CC0; do not ship in the final look

6. Week-by-week plan

Hours are for planning; adjust after Week 1.

Week 0 (3–4 days): foundations

- Reference board, Look Bible (section 3) and standards document (section 4).
- Create `docs/environment-kit/catalog.csv` with columns: ID, name, region, [K]/[V]/[D], priority, tri budget, status, licence, region lock.
- Blender template with units, collections, export preset and the 2 m grid.
- Capture **baseline screenshots** of Q01 and L04 from 6 fixed camera positions each (you already have `trees-shot.mjs` and the browser harness). These become the before/after comparison.
- **Done when:** documents are committed and 12 baseline images exist.

Week 1: pipeline spike (about 20 h) — Gate 1

- Build `vendor/witness-loaders.js` with esbuild. The loaders must use the *same* Three.js instance as the engine. Alias `three` to a shim that reads `globalThis.__THREE`, and set that global where `witness-world.js` receives `T` in the `AdventureVisuals.sky` hook.
- Add a heightfield sampler: render the ground height into a Float32 grid once per scene and replace the per-object ray casts.
- Write a scene manifest format (`scenes/q01.kit.json`): piece ID, position, rotation, scale, variant, seed.
- Place pieces with `InstancedMesh` or `BatchedMesh`; read `extras.footprint` into `solids`.
- Update `build.cjs` and the service worker: packs by region (`packs/cae/…`), lazy-loaded and cached at runtime, not pre-cached at install. Keep a procedural fallback so the standalone offline HTML still runs without packs.
- Test with one throw-away asset: a wall module instanced 200 times.
- **Gate 1 (pass/fail):** wall instanced ×200 with correct collision, KTX2 texture, offline load works, `npm test` green, no frame-time regression on your target laptop.

Week 2: materials and terrain

- Build the **material library** (12 PBR sets, 1K–2K): Herodian limestone, kurkar sandstone, basalt dry-stone, lime plaster (clean, cracked), mud-straw roof, weathered wood, rope/linen, terracotta, packed-earth path, dry terra-rossa soil, limestone outcrop, beach sand. Start from Poly Haven / ambientCG, then colour-grade to the palette.

- **Terrain shader:** 4-layer splat blend with height-based blending, triplanar projection on steep faces, macro colour variation to hide tiling, painted path mask.
- Author terrain relief in Blender or from a 16-bit heightmap: low hills, a worn track, a shoreline that connects to the existing `carveSeabed`.
- **Done when:** the ground reads as real soil and stone at 1.7 m eye height and shows no visible tiling at 30 m. This alone removes the flat gradient look.

Week 3: Caesarea architecture, part 1 (P0)

- Wall modules (2 m, 4 m, corner, end cap, plinth, cornice) with the Herodian boss-and-margin detail: geometry bevel at edges plus normal-map detail, 8 cm margin.
- Stone quay edge with mooring stones; steps; arched gate; door and window frames; flat roof with parapet.
- Three damage variants per wall via vertex-paint and decal swaps.
- Weekly review: place pieces in the Q01 scene next to the old boxes and compare against the baseline images.

Week 4: Caesarea architecture, part 2 and Galilee architecture (P0)

- Column set (base, drum, capital), colonnade module, **barrel-vault warehouse module** (harbor warehouses).
- **Basalt set for Galilee:** rough dry-stone walls, courtyard wall, doorway, roof stairs, thatch-and-mud roof, oven, quern stones, fish-drying racks, the basalt shore quay.
- Decals: dirt streaks, salt stains, cracks, moss, chisel marks via `DecalGeometry`.
- **Gate 2 (end of Week 4):** the first look-dev shot of Q01 is lit and compared with the cover-art reference. If it does not read as "clearly better than baseline", stop adding pieces and fix lighting, materials and wear first.

Week 5: nature kit (P0)

- **Rocks:** 12 variants (boulders, stratified limestone, cliff faces, scree) plus 3 cave-mouth pieces for L02 later.
- **Vegetation (Levant palette):** olive, terebinth/oak, fig, date palm, tamarisk, reeds and papyrus (Galilee shore), thorn shrubs, dry grass clumps, three wildflowers. LOD0 + LOD1 + billboard.
- Wind shader driven by the vertex-colour blue channel; density maps for scattering instead of hand placement.
- Improve grass: more instances, dry/green variants, distance fade.
- **Gate 3 (end of Week 5):** go/no-go to continue the full plan or cut to the one-month version (section 11).

Week 6: props and hero assets (P0 / P1)

- **Hero assets (≤ 12 k tris):** a planked Kinneret-style fishing boat, the Caesarea lighthouse, the cart, a goat-hair tent, a windlass.
- **Small props:** amphorae (three types), oil lamps, baskets, rope coils, nets, crates, braziers, anchor stones, benches, tables, awnings.
- Replace `ginosarBoat()`, `boat()`, `cart()`, `jar()`, `lamp()` and `caesareaLighthouse()` with kit pieces and keep their coordinates unchanged so craft tests still pass.

Week 7: integration, lighting and optimisation (P0)

- Dress Q01 and L04 from the manifest.
- Lighting pass: sun and IBL against the new materials, cascaded or follow-camera shadows, AO, per-scene colour grade, height fog.
- LOD tuning, texture mip budgets, instancing audit, load-time profile.
- "Balanced" quality tier: smaller textures, fewer instances, no decals.
- Update tests; add `test/kit-budget.cjs` that fails the build if any pack breaks the section 4 budgets.

Week 8: polish, QA, documentation — Gate 4

- Visual regression pass using the 12 baseline cameras plus new ones.
- Playtest with 3–5 people (your catechist group counts): note first impressions and any motion or performance complaints.
- Update `ART-CREDITS.md`, the catalog and the standards document.
- **Gate 4:** decide whether to dress Jerusalem and the Legends next, or to invest in characters.

7. Asset catalog (priority checklist)

Priority	Group	Items (approx. count)
P0	Materials	12 PBR sets
P0	Terrain	Splat shader, 2 scene heightmaps, path masks
P0	Caesarea architecture	~25 modules incl. barrel-vault warehouse, quay, colonnade
P0	Galilee architecture	~15 modules incl. courtyard, stairs, thatch roof, basalt quay
P0	Nature	12 rocks, 10 plants, grass system
P0	Hero props	Boat, lighthouse, cart, tent, windlass
P1	Small props	~25 pieces
P1	Decals	~20 (dirt, stains, cracks, moss, marks)
P2	Jerusalem street set	~15 modules
P2	Cistern and prison interiors	~15 modules (replaces the open cylinder in L03 and the box prison in L05)
P2	En Gedi cave pieces	~6

Every row also carries its [K]/[V]/[D] tag and region lock in `catalog.csv`.

8. Integration architecture

- **Loader bundle.** `witness-loaders.js` exposes `WitnessKit.load(packId)`. It uses the engine's own Three.js instance so `instanceof` checks and materials remain compatible. Pin to `three@0.187.0`.
- **Manifest-driven scenes.** `decorate()` reads the kit manifest and places pieces; existing procedural builders remain as fallbacks for scenes not yet converted.
- **Heightfield.** One sampler per scene replaces ray casts.
- **Collision.** Footprints from glTF `extras` feed `solids`. Never add solids to NPC or prop anchors (the code already warns about this).
- **Materials.** Authored PBR maps are primary. `AdventureVisuals.surface` is kept only to add dust and wetness overlays that respond to the biome.
- **Packs.** `packs/<region>.glb` plus textures; lazy-loaded with a progress bar; stored by runtime caching in the service worker; the standalone HTML build uses a "lite" procedural fallback.
- **Shadows.** Move from one 2048² map over 68 × 61 m to a follow-player frustum or cascades once modules are in.

9. QA gates and metrics

- **Functional:** `npm test`, `npm run test:browser`, `npm run test:craft-details` pass after every merge.
- **Visual regression:** 12 baseline cameras plus new ones; compare weekly; keep a "before/after" folder for your portfolio and pitch.
- **Performance:** record frame time at fixed camera points on your reference desktop and a mid phone; target a stable 60 fps desktop and 30 fps mobile balanced.
- **Budget checks:** automated by `test/kit-budget.cjs` (triangles, texture sizes, material counts, file sizes per pack).
- **Licence audit:** every external asset has a ledger row before it is committed.

10. Risks and mitigations

Risk	Mitigation
Scope creep	Only two scenes are in scope; Gates 1–4 can cut work
Modelling takes longer than planned	Kitbash CC0 pieces first, then re-skin; use Geometry Nodes generators for stone and thatch
Loader and Three.js version mismatch	Pin to 0.187.0 and use the shim; test with a trivial asset in Week 1
Offline HTML build bloats	Packs are lazy-loaded; standalone build ships a lite fallback
Performance on phones	Balanced tier with smaller textures, fewer instances, no decals
Historical or tonal inconsistency	Look Bible, region locks, [K]/[V]/[D] tags

Risk	Mitigation
License trouble	Prefer Poly Haven and ambientCG; ledger every asset; read Fab EULA before using scans
Breaking gameplay coordinates	Keep object positions fixed; run craft tests on every change

11. Decision gates and the one-month cut

- **Gate 1 (end of Week 1):** pipeline works end to end. If not, fix it before any art.
- **Gate 2 (end of Week 4):** the Q01 look-dev shot is clearly better than baseline.
- **Gate 3 (end of Week 5):** continue to the full 8 weeks, or cut.
- **Gate 4 (end of Week 8):** choose what to dress or build next.

One-month version (if time is short). Do Weeks 0–2 as written. Then build only the Caesarea P0 architecture subset (walls, quay, columns, lighthouse), 6 rocks, 5 plants and 3 hero props for **Q01 only**. Skip Galilee until the second month. You still get a convincing first scene.

12. Next three days

1. Create `docs/environment-kit/` with the Look Bible, standards and an empty `catalog.csv`.
2. Run your existing browser test harness and capture the 12 baseline screenshots.
3. Start the Week 1 spike: add `witness-loaders.js`, expose `globalThis.__THREE`, and load one test GLB into Q01 to confirm the Three.js instance matches.